ConsensusCode-Evolve: Consensus-Aware Generative Policy Search for Multi-Agent Inventory Control

Author Name

***Abstract*—Generative AI is creating new opportunities for industrial automation, but direct deployment of large language models (LLMs) as online controllers remains difficult because of cost, safety, interpretability, and reproducibility concerns. This paper proposes *ConsensusCode-Evolve*, a simulator-grounded framework that uses an LLM as an offline code-evolution engine for multi-agent inventory automation. This paper examines the problem of multi-location replenishment and transhipment, a typical task in discrete manufacturing and supply chain management. In this context, local inventory agents must coordinate material flows while considering demand uncertainty, lead times, and capacity constraints. The paper also formulates a simulator benchmark for this setting, since existing studies cover related inventory and supply-chain control problems but do not provide a widely used benchmark for the full setting considered here. Instead of asking an LLM to control the system online, the proposed method asks it to generate constrained Python policy functions for replenishment, transshipment proposal, and consensus gating. Candidate policies are validated by an abstract-syntax-tree safety checker, evaluated in the simulator, and selected through an archive-based loop using compact numerical feedback. A consensus-seeking layer models donor willingness, receiver urgency, and fairness debt, so transshipment actions are accepted only when they satisfy both local and system-level requirements. The resulting policies are lightweight, executable without an LLM at deployment time, and directly inspectable by engineers. The paper proposes the benchmark problem, simulator, method, safety constraints, evaluation protocol, and compares the proposed method against classical inventory policies, genetic programming, and learning-based baselines. The central hypothesis is that simulator-grounded generative code search can discover interpretable, consensus-aware inventory policies with lower token cost than online LLM agents while retaining automation-oriented robustness and deployability.**

***Index Terms*—Generative AI, industrial automation, inventory control, code generation.**

I. Introduction

Industrial automation increasingly requires decision-making systems that coordinate distributed assets, adapt to uncertain operating conditions, and remain interpretable to engineers [1]. In discrete manufacturing and supply operations, material-flow decisions such as replenishment, inventory allocation, and transshipment are not merely numerical optimization tasks. They involve multiple sites or production units with local constraints, asymmetric costs, and sometimes conflicting interests [2]. A site with surplus inventory may be asked to support another site, but such a transfer can be harmful if the donor later faces a demand surge. This makes multi-site inventory automation a natural setting for studying distributed decision-making and consensus-aware control [3].

Similar decision settings arise in many supply chains. Large retailers must balance stock across stores, regional warehouses, and distribution centers [4]. E-commerce firms operate fulfillment networks where local shortages and overstock can occur at the same time. Manufacturers coordinate raw materials, components, and finished goods across plants. Spare-parts networks in transportation, energy, and equipment maintenance must avoid service failures while limiting slow-moving inventory. Pharmaceutical, medical, grocery, and fresh-product supply chains face a further tension: shortages can be costly, but excess stock can also lead to waste, expiration, or high holding cost. These examples differ in detail, but they share the same core difficulty: local inventory decisions affect the whole network [5].

Large language models (LLMs) [6] create a new way to generate decision logic, but direct online use of an LLM as an industrial controller is difficult. Online calls add cost and latency. They also make reproducibility, safety review, and engineering inspection harder. Inventory control adds further constraints: decisions must respect material availability, capacity, lead times, order costs, and transfer costs. *A useful generative method for this setting should therefore be grounded in the simulator or plant model, should produce deterministic policies for deployment, and should expose the decision logic in a form that engineers can inspect.*

Recent progress in LLM-based code generation and program search suggests one promising path. AlphaCode showed that large language models can generate many candidate programs and use selection mechanisms to solve competitive programming problems [7]. CodeRL incorporated execution feedback into code generation [8]. Self-Refine and Reflexion demonstrated that language models can iteratively improve outputs using compact feedback or verbal memory [9], [10]. Voyager used LLM-generated executable code as reusable skills in an embodied environment [11], and Eureka showed that LLMs can write reward functions evaluated through simulation [12]. FunSearch is particularly relevant because it combines LLM-generated programs with automatic evaluation to discover high-performing mathematical constructions [13]. These studies suggest that an LLM need not be the controller itself; instead, it can serve as a generator of candidate executable programs, while an external evaluator provides grounding.

This paper transfers that idea to industrial inventory automation. We propose *ConsensusCode-Evolve*, a token-efficient framework in which an LLM evolves executable, safe Python

policy functions offline. Each candidate policy contains three functions: a *replenishment policy*, a *transshipment proposal policy*, and a *consensus gate*. The replenishment function determines local order quantities. The transshipment proposal function suggests pairwise material movements. The consensus gate accepts, scales, or rejects proposed transfers based on donor willingness, receiver urgency, network benefit, and fairness debt. Candidate code is statically checked before execution and then evaluated in a multi-site inventory simulator. Only compact performance summaries and selected policy code snippets are returned to the LLM for the next mutation round.

The proposed approach differs from direct LLM planning in three ways. First, the LLM is used offline, which reduces token cost and avoids dependence on a large model during deployment. Second, generated code is constrained to a executable safe subset of Python, making the final policies inspectable and reproducible. Third, the method embeds autonomous multi-agent consensus seeking into the action layer, so transshipment decisions are not treated as forced centralized commands but as negotiated actions among local inventory agents. The main contributions of this work are summarized as follows:

- We formulate a multi-site inventory control problem with replenishment and lateral transshipment under uncertain demand, local site states, capacity limits, cost asymmetry, and partial coordination. We also develop a configurable simulator benchmark with common transition rules, feasibility checks, and cost accounting.
- We propose *ConsensusCode-Evolve*, a simulator-grounded generative code search framework for learning executable inventory policies. Instead of directly optimizing actions for a single instance, the framework evolves policy code that can be evaluated, selected, and reused across simulated inventory scenarios.
- We design a consensus-aware policy representation that decomposes decision-making into replenishment, transshipment proposal, and agreement gating. This structure makes the generated policy code better aligned with the coupled nature of multi-site inventory decisions, where one site's transfer action may affect the shortage risk and cost of another site.
- We conduct a set of experiments comparing the evolved policies with classical inventory control rules, genetic programming, and learning-based baselines. The results show the effectiveness of the proposed method from several aspects, including cost, service level, transfer behavior, fairness, and robustness under different demand and site configurations.

II. Problem Definition and Simulator

This paper studies a replenish-and-transship benchmark problem for multi-site inventory management. The system contains several inventory sites that face uncertain, time-varying demand. Each site is treated as a local inventory agent with its own stock level, cost parameters, capacity, pipeline information, and demand forecasts. At each decision period, the agents jointly determine replenishment quantities and lateral transshipment quantities. The purpose is to reduce inventory operating cost while keeping transfers feasible and avoiding policies that repeatedly expose the same donor site to shortage risk.

TABLE I
Notation

Symbol	Description
N	Number of inventory sites
T	Number of decision periods in one simulation episode
i, j	Site indices, with $i, j \in \{1, \dots, N\}$
$D_i(t)$	Random demand at site i in period t
$\hat{d}_{i,t}^{(\kappa)}$	The κth demand forecast feature for site i at period t
K	Number of demand forecast features exposed to the policy
L_o	Replenishment lead time
$I_i(t)$	On-hand inventory at site i at the start of period t
$P_i(t)$	Pipeline order information available at site i
Cap_i	Storage capacity of site i
U_i	Production-capacity parameter exposed to the policy state
$x_i(t)$	Replenishment quantity ordered for site i in period t
$q_{ij}(t)$	Lateral transshipment quantity between sites i and j
a_i	Per-unit ordering cost at site i
k_i	Fixed ordering cost at site i
h_i	Holding cost per positive inventory unit at site i
b_i	Lost-sales cost per shortage unit at site i
c^{tr}	Per-unit transshipment cost
K^{tr}	Fixed cost charged when a transshipment is made
$S(t)$	State observed by the policy at period t
C_t	Total cost incurred in period t
π	Inventory policy used to choose replenishment and transshipment actions

The simulator is part of our contribution. To the best of our knowledge, no existing benchmark or simulator systematically defines this multi-site inventory problem, although the problem is important in many real-world supply chains. The proposed simulator provides a common training and test environment for policies that must make replenishment and lateral transshipment decisions under the same demand process, transition rules, feasibility checks, and cost accounting. It is useful because closed-form analysis is difficult when demand is uncertain, sites have different local states, decisions are coupled through transfers, and a local action can change the shortage risk of another site. The notation used in this section is summarized in Table I, and the simulator design is summarized in Table II.

The scenario is shown at a high level in Fig. 1. The figure is meant to make the benchmark structure clear: the simulator provides site states, forecasts, and feedback; local inventory agents choose replenishment and transshipment actions; and the proposed method generates the executable decision logic that is tested in the simulator.

A. System State and Demand

We consider a finite-horizon periodic-review system with N sites and decision periods $t = 1, \dots, T$. The current simulator supports the two-site and three-site cases. At the beginning of

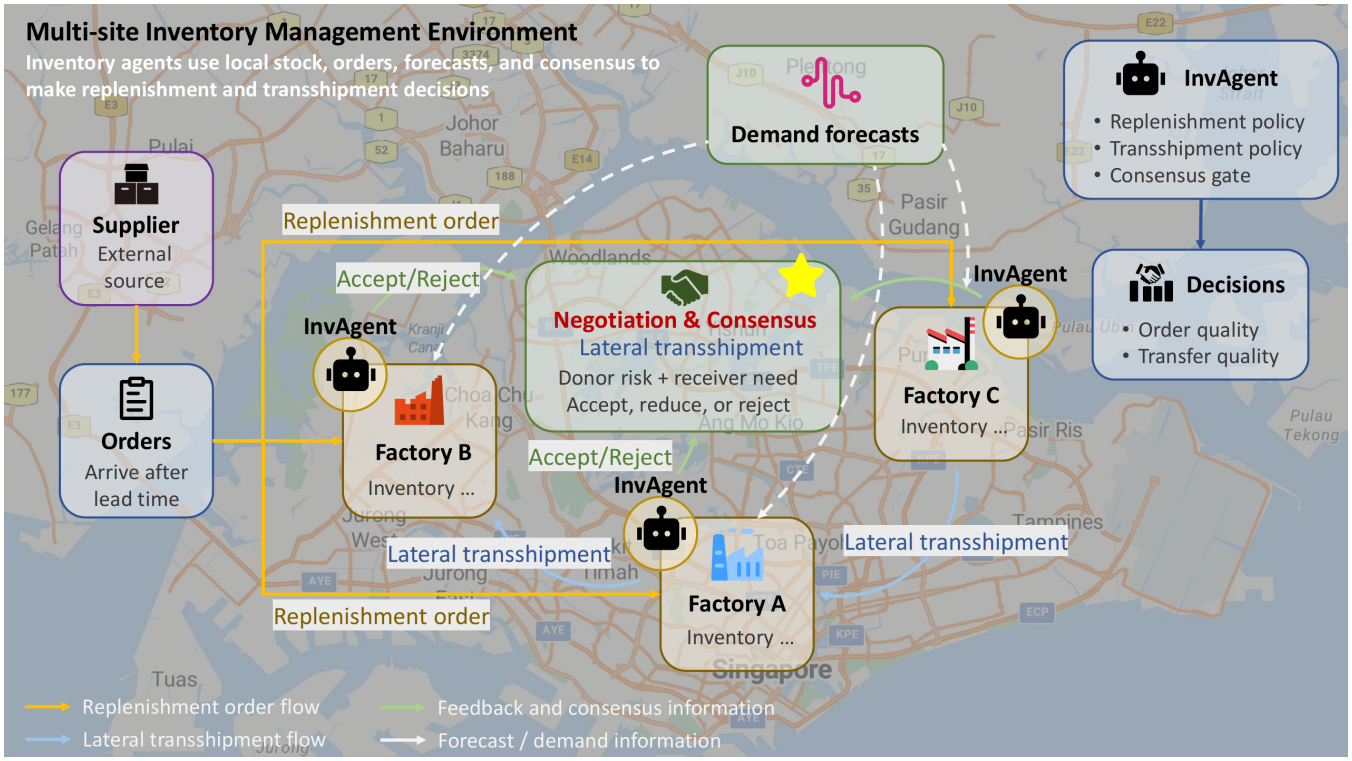


Fig. 1. Illustrative multi-site inventory environment using Singapore as an example geography. Each site represents a local factory, warehouse, or inventory location operated by an inventory-control agent that observes local inventory state and demand forecasts. The simulator updates demand, replenishment lead times, lateral transshipment, feasibility checks, and cost accounting. The proposed method generates executable policy logic for these agents, and candidate policies are evaluated using cost, service, and coordination measures.

TABLE II
SIMULATOR BENCHMARK OVERVIEW

Component	Benchmark specification
Inputs	Number of sites, episode length, initial inventory, capacity, production-capacity parameter, ordering costs, holding costs, lost-sales costs, transshipment costs, demand model, and forecast features
State	Site-level inventory, costs, capacity, pipeline order information, forecast vector, and pairwise transshipment states
Actions	Pairwise signed transshipment quantities and site-level replenishment quantities
Demand	Synthetic stochastic demand or loaded demand records; synthetic demand uses time-varying Poisson means
Transition	Order arrival, demand realization, lateral transshipment, cost calculation, pipeline update, and forecast update
Costs	Transshipment cost, holding and lost-sales cost, and order cost
Evaluation	Average cost per episode plus service and coordination metrics, including stockout proxy, accepted/rejected transfers, and fairness debt
Configurability	Site number, demand scale, cost asymmetry, capacity, initial inventory, lead time, episode length, and benchmark scenario

a decision period, the policy observes a state $S(t)$ containing local site information and pairwise transshipment information. For each site, the local information includes current inventory, holding cost, lost-sales cost, storage capacity, production-capacity parameter, fixed and unit ordering costs, pipeline order information, a demand forecast vector, and transshipment cost parameters. The forecast vector is written as

$$\hat{\mathbf{d}}_{i,t} = \left[\hat{d}_{i,t}^{(1)}, \hat{d}_{i,t}^{(2)}, \ldots, \hat{d}_{i,t}^{(K)}\right], \tag{1}$$

where K is the number of forecast features exposed to the policy. The general formulation allows any K. The current experiments use $K = 2$, matching the two forecast values exposed by the implemented simulator state.

For inventory scenarios, the simulator first samples a time-varying demand mean for each site and period. The realized demand $D_i(t)$ is then sampled from a Poisson distribution with that period-specific mean. The forecast features are generated from the demand model and provide short-horizon demand information to the policy. For test scenarios based on historical demand records, the simulator uses the provided demand sequence and uses the same sequence as the forecast record. In both cases, future realized demand is not known when the policy chooses the action.

B. Decision Variables and Action Structure

At each period, the policy chooses a replenishment vector and a transshipment vector. The replenishment vector is

$$\mathbf{x}(t) = [x_1(t), \ldots, x_N(t)], \tag{2}$$

where $x_i(t)$ is the order quantity for site i. Orders are not available immediately. They enter the pipeline and arrive after the replenishment lead time L_o.

The transshipment vector contains one signed quantity for each unordered site pair. For a pair (i, j) with $i < j$, $q_{ij}(t) > 0$

denotes a transfer from site i to site j, while $q_{ij}(t) < 0$ denotes a transfer from site j to site i. For two sites, the action contains one transshipment quantity. For three sites, the action contains quantities for pairs $(1,2)$, $(1,3)$, and $(2,3)$.

The complete action is therefore

$$\mathbf{d}(t) = (\mathbf{q}(t), \mathbf{x}(t)), \tag{3}$$

where the transshipment part appears before the replenishment part in the simulator action vector.

C. Inventory Transition

For each site, an outstanding order arrives at the start of the simulator step. The inventory after order arrival is capped by storage capacity:

$$\bar{I}_i(t) = \min\{Cap_i, I_i(t) + A_i(t)\}, \tag{4}$$

where $A_i(t)$ is the pipeline order that arrives in period t. Demand is then realized and subtracted:

$$\tilde{I}_i(t) = \bar{I}_i(t) - D_i(t). \tag{5}$$

After this demand update, lateral transshipments are applied. A proposed transfer is executed only if the donor has enough available inventory at the point where the transfer is checked. If the donor inventory is insufficient, the simulator sets that transfer to zero. Let $Q_i(t)$ denote the net inventory change at site i caused by all accepted transshipments in period t. The inventory after transshipment is

$$I_i^+(t) = \tilde{I}_i(t) + Q_i(t). \tag{6}$$

If $I_i^+(t)$ is positive, the site pays holding cost. If $I_i^+(t)$ is negative, the shortage is charged as lost sales and the inventory carried to the next period is reset to zero:

$$I_i(t+1) = \max\{0, I_i^+(t)\}. \tag{7}$$

The new replenishment decision $x_i(t)$ is then stored in the pipeline and becomes available after the lead time.

D. Feasibility Constraints

The simulator enforces simple feasibility rules. Replenishment quantities are nonnegative after policy adaptation and are capped before entering the pipeline. In the current simulator step, any replenishment quantity above site capacity is reduced to Cap_i:

$$0 \le x_i(t) \le Cap_i. \tag{8}$$

For LLM-generated policies, the adapter first clips order quantities to a nonnegative range before the simulator applies its own capacity cap.

Transshipment feasibility is checked through donor availability. For each proposed pairwise transfer, the donor must have enough inventory to cover the requested movement after prior transfers in the same period have been considered. Otherwise, that pairwise movement is not executed. This rule prevents the simulator from creating negative donor inventory through transshipment alone, although a site may still have negative inventory after demand if it faces shortage before receiving enough transfer support.

E. Cost and Objective

The one-period cost contains three parts: transshipment cost, holding and lost-sales cost, and order cost. The transshipment cost is

$$C_t^{tr} = \sum_{(i,j)} \left(c^{tr} |q_{ij}^{acc}(t)| + K^{tr} \mathbb{I}\{|q_{ij}^{acc}(t)| > 0\} \right), \tag{9}$$

where $q_{ij}^{acc}(t)$ is the accepted transfer after feasibility checking. The holding and lost-sales cost is

$$C_t^{hl} = \sum_{i=1}^{N} \left(h_i \max\{I_i^+(t), 0\} + b_i \max\{-I_i^+(t), 0\} \right). \tag{10}$$

The order cost is

$$C_t^{ord} = \sum_{i=1}^{N} \left(a_i x_i(t) + k_i \mathbb{I}\{x_i(t) > 0\} \right). \tag{11}$$

The total period cost is

$$C_t = C_t^{tr} + C_t^{hl} + C_t^{ord}. \tag{12}$$

Given a policy π, the simulator evaluates the average cost over one episode:

$$J_T(\pi) = \frac{1}{T} \sum_{t=1}^{T} C_t. \tag{13}$$

The optimization goal is to find a policy with low expected average cost across demand realizations and test scenarios:

$$\min_{\pi} \ \mathbb{E}\left[J_T(\pi) \right]. \tag{14}$$

The evaluation also records service and coordination measures, including stockout proxy count, accepted transfer count, rejected transfer count, and fairness debt. Fairness debt is computed in the current prototype as the sum of absolute net pairwise transfers over the episode. These measures do not replace the cost objective. They are used to judge whether a low-cost policy achieves its performance through reasonable coordination behavior.

F. Benchmark Scope

The benchmark is designed for researchers who need a repeatable environment for multi-site, multi-agent inventory control with lateral transshipment. It differs from a single-site inventory benchmark because decisions at one site can change the shortage risk at another site. It differs from a standard multi-echelon benchmark because the sites considered here are at the same echelon and can share inventory laterally. It also differs from a pure ordering benchmark because a policy must decide both replenishment and pairwise transfer quantities.

The simulator can be used to compare hand-designed rules, GP policies, DRL policies, LLM-generated code policies, and other policy-search methods under the same demand and cost setting. Researchers can configure the number of sites supported by the simulator, demand scale, cost asymmetry, initial inventory, capacity, lead time, and episode length. The present implementation supports two-site and three-site replenish-and-transship scenarios; extending the simulator to larger networks is a natural next step.

REFERENCES

[1] K. Zhang, Y. Shi, S. Karnouskos, T. Sauter, H. Fang, and A. W. Colombo, "Advancements in industrial cyber-physical systems: An overview and perspectives," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 1, pp. 716–729, 2022.

[2] N. Agrawal and S. A. Smith, "Multi-location inventory models for retail supply chain management: A review of recent research," *Retail Supply Chain Management: Quantitative Models and Empirical Studies*, pp. 207–235, 2008.

[3] K. Sampath, S. Nishad, S. K. R. Danda, P. Dayama, and S. Sankagiri, "Inventory pooling using deep reinforcement learning," in *2022 IEEE International Conference on Services Computing (SCC)*. IEEE, 2022, pp. 259–267.

[4] J. J. Vicente, S. Relvas, and A. P. Barbosa-Póvoa, "Distribution and inventory planning in multi-echelon supply chains under demand uncertainty," *International Journal of Operational Research*, vol. 46, no. 1, pp. 93–117, 2023.

[5] A. Ala, A. Goli, S. Mirjalili, and V. Simic, "A fuzzy multi-objective optimization model for sustainable healthcare supply chain network design," *Applied Soft Computing*, vol. 150, p. 111012, 2024.

[6] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang *et al.*, "A survey on evaluation of large language models," *ACM transactions on intelligent systems and technology*, vol. 15, no. 3, pp. 1–45, 2024.

[7] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, T. Hubert, P. Choy, C. de Masson d'Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Gowal, A. Cherepanov, J. Molloy, D. Mankowitz, E. Sutherland Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[8] H. Le, Y. Wang, A. D. Gotmare, S. Savarese, and S. C. H. Hoi, "CodeRL: Mastering code generation through pretrained models and deep reinforcement learning," in *Advances in Neural Information Processing Systems*, 2022.

[9] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Welleck, B. P. Majumder, S. Gupta, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems*, 2023.

[10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, 2023.

[11] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint arXiv:2305.16291*, 2023.

[12] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, "Eureka: Human-level reward design via coding large language models," in *International Conference on Learning Representations*, 2024.

[13] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, pp. 468–475, 2024.

[14] P. H. Zipkin, *Foundations of Inventory Management*. McGraw-Hill, 2000.

[15] S. Axsater, *Inventory Control*, 3rd ed. Springer, 2015.

[16] C. Paterson, G. Kiesmuller, R. Teunter, and K. Glazebrook, "Inventory models with lateral transshipments: A review," *European Journal of Operational Research*, vol. 210, no. 2, pp. 125–136, 2011.

[17] K. Kogan, S. Lou, C. S. Tapiero, and M. Shnaiderman, "Supply chain with inventory review and dependent demand distributions: Dynamic inventory outsourcing," *IEEE Transactions on Automation Science and Engineering*, vol. 7, no. 2, pp. 197–207, 2010.

[18] M. Boccadoro, F. Martinelli, and P. Valigi, "Supply chain management by H-infinity control," *IEEE Transactions on Automation Science and Engineering*, vol. 5, no. 4, pp. 703–707, 2008.

[19] A. Castelnuovo, L. Ferrarini, and L. Piroddi, "An incremental petri net-based approach to the modeling of production sequences in manufacturing systems," *IEEE Transactions on Automation Science and Engineering*, vol. 4, no. 3, pp. 424–434, 2007.

[20] L. Liu, C.-B. Yan, and J. Li, "Modeling, analysis, and improvement of batch-discrete manufacturing systems: A systems approach," *IEEE Transactions on Automation Science and Engineering*, vol. 19, no. 3, pp. 1567–1585, 2022.

[21] C. Zhang, Q. Gao, M. V. Basin, J. Lu, and H. Liu, "Robust control of multi-line re-entrant manufacturing plants via stochastic continuum models," *IEEE Transactions on Automation Science and Engineering*, vol. 21, no. 4, pp. 4923–4935, 2024.

[22] I.-B. Park, J. Huh, J. Kim, and J. Park, "A reinforcement learning approach to robust scheduling of semiconductor manufacturing facilities," *IEEE Transactions on Automation Science and Engineering*, pp. 1–12, 2020.

[23] S. Luo, L. Zhang, and Y. Fan, "Real-time scheduling for dynamic partial-no-wait multiobjective flexible job shop by deep reinforcement learning," *IEEE Transactions on Automation Science and Engineering*, vol. 19, no. 4, pp. 3020–3038, 2022.

[24] X. Wang, Y. Laili, L. Zhang, and Y. Liu, "Hybrid task scheduling in cloud manufacturing with sparse-reward deep reinforcement learning," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 1878–1892, 2025.

[25] M. C. Fu, "Simulation optimization," in *Handbook of Simulation Optimization*. Springer, 2015.

[26] S. Amaran, N. V. Sahinidis, B. Sharda, and S. J. Bury, "Simulation optimization: A review of algorithms and applications," *Annals of Operations Research*, vol. 240, no. 1, pp. 351–380, 2016.

[27] X. Shen, S. Liu, B. Zhou, T. Wu, Q. Zhang, and J. Bao, "Digital twin-driven reinforcement learning method for marine equipment vehicles scheduling problem," *IEEE Transactions on Automation Science and Engineering*, vol. 21, no. 3, pp. 2173–2183, 2024.

[28] C.-Y. Lin, T.-L. Tseng, and T.-H. Tsai, "A digital twin framework with deep feature extraction and gaussian process for multi-objective optimization in semiconductor manufacturing," *IEEE Transactions on Automation Science and Engineering*, vol. 23, pp. 1928–1949, 2026.

[29] J. R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, 1992.

[30] R. Poli, W. B. Langdon, and N. F. McPhee, *A Field Guide to Genetic Programming*. Lulu, 2008.

[31] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[32] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[33] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[34] R. Olfati-Saber, J. A. Fax, and R. M. Murray, "Consensus and cooperation in networked multi-agent systems," *Proceedings of the IEEE*, vol. 95, no. 1, pp. 215–233, 2007.

[35] C. Wang, "Requirement-based bidding language for agent-based scheduling," *IEEE Transactions on Automation Science and Engineering*, vol. 10, no. 1, pp. 219–223, 2013.

[36] W. Wei, C. Jiang, and Y. Huang, "A data-driven human-machine collaborative product design system toward intelligent manufacturing," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 736–749, 2025.

[37] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," *arXiv preprint arXiv:2309.03409*, 2023.

[38] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel, "Promptbreeder: Self-referential self-improvement via prompt evolution," *arXiv preprint arXiv:2309.16797*, 2023.

[39] S. K. Chaharsooghi, J. Heydari, and S. H. Zegordi, "A reinforcement learning model for supply chain ordering management: An application to the beer game," *Decision Support Systems*, vol. 45, no. 4, pp. 949–959, 2008.

[40] C. D. Hubbs, H. D. Perez, O. Sarwar, N. V. Sahinidis, I. E. Grossmann, and J. M. Wassick, "OR-gym: A reinforcement learning library for operations research problems," *arXiv preprint arXiv:2008.06319*, 2020.

[41] X. Liu, M. Hu, Y. Peng, and Y. Yang, "Multi-agent deep reinforcement learning for multi-echelon inventory management," *Production and Operations Management*, vol. 34, no. 7, pp. 1836–1856, 2024.

[42] J. Chen, "Multi-location inventory management under time-varying demands: A deep reinforcement learning approach," in *Proceedings of the International Conference on Industrial Engineering and Operations Management*, 2024.

[43] A. J. Clark and H. Scarf, "Optimal policies for a multi-echelon inventory problem," *Management Science*, vol. 6, no. 4, pp. 475–490, 1960.

[44] D. Bertsimas and A. Thiele, "A robust optimization approach to inventory theory," *Operations Research*, vol. 54, no. 1, pp. 150–168, 2006.

[45] A. A. Syntetos, M. Z. Babai, J. E. Boylan, S. Kolassa, and K. Nikolopoulos, "Supply chain forecasting: Theory, practice, their gap and the future," *European Journal of Operational Research*, vol. 252, no. 1, pp. 1–26, 2016.